Networks & Distributed Programming

The Application Layer: HTTP

Lecture 4



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

Application Layer
Web and HTTP

- Until the early 1990s the Internet was used **primarily by researchers**, **academics**, and university **students**
  - To log in to remote hosts (Telnet), to transfer files (FTP), to receive and send news, to receive and send electronic mail.

- Although these applications were (and continue to be) extremely useful, the Internet was essentially unknown outside of the academic and research communities.

- In the early 1990s the **World Wide Web** was the **first Internet application** that caught the general public.

Application Layer

Web and HTTP

- The **World Wide Web** (WWW or simply Web) is a **collection of information** such as documents, images, video, audio, etc. that can be accessed over the Internet according to a specific protocol called **HyperText Transfer Protocol** (HTTP).

- HTTP-based communication is typically **client-server**:
  - A **client program** that translates users' requests into HTTP messages. This is implemented into the **web browsers** (Chrome, Firefox, Edge, Safari, etc.).
  - A **server program** that executes the **HTTP request** and returns a **HTTP response**.

- HTTP mainly defines:
  - the **structure of messages**
  - how **client and server exchange** such messages.

- The Web and its protocols **serve as a platform** for web-based applications such as YouTube, Web-based e-mail (e.g., Gmail), and most mobile Internet applications, including Social Networks.

Application Layer
URL

- The information on the web are called **resources** (or objects) and are identified by a **Uniform Resource Locator** (URL) which is a string composed as follows:

  [protocol]://[usrinfo@][host][:port][/path][?query][#fragment]

- Where:
  - [protocol] is the **protocol** to use while accessing the resource (HTTP, HTTPS, FTP, etc.).
  - [usrinfo] (optional) are **user's information** such as username and password followed by a @ (e.g., username:password@). This is **mostly deprecated** due to security issues.
  - [host] is the **name or the IP** address of the server.
  - [port] (optional) is the **port** to use (often inferred from the protocol).
  - [path] is the **path of the resource** within the server (e.g., /path/to/resource).
  - [query] is preceded by the ? and specifies **possible requests**.
  - [fragment] preceded by the # **identifies an element** in the resource (e.g., a form).

Application Layer
URL

- Most Web pages consist of a **base HTML** (HyperText Markup Language) file and several references to additional **objects** (images, videos, Java applets, etc.).
  - Example: a Web page containing HTML text and five JPEG images **has six objects**: the base HTML file plus the five images.

- An example of URL is:

  http://www.someSchool.edu/someDepartment/picture1.gif

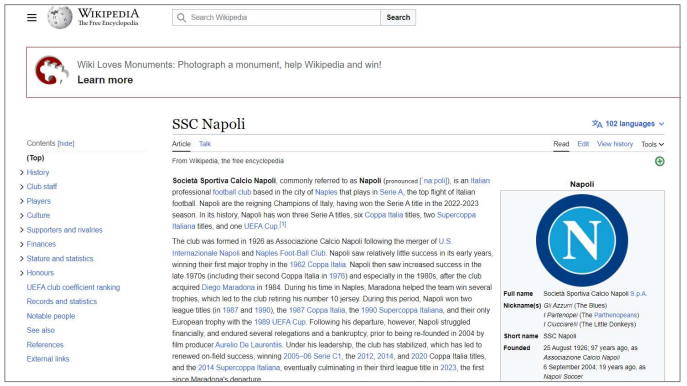
- Where:
  - *http* is the **protocol**.
  - *www.someSchool.edu* is the **hostname**.
  - */someDepartment/picture1.gif* is the **path** to the object.

Application Layer
URL

- Real example of URL query:

https://en.wikipedia.org/w/index.php?title=SSC_Napoli

- Where:
  - *https* is the **protocol**.
  - *en.wikipedia.org* is the **hostname**.
  - */w/index.php* is the **path** to the page.
  - *?title=SSC_Napoli* is the **query**.



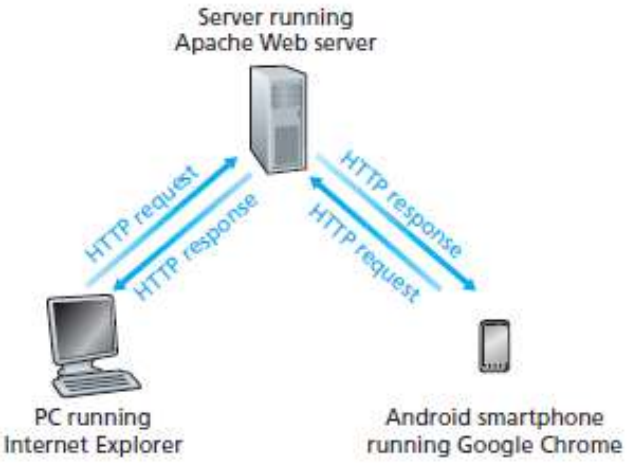
en.wikipedia.org/wiki/SSC_Napoli

- In this case, it is the same as asking for the page:

https://en.wikipedia.org/wiki/SSC_Napoli

Application Layer
HTTP

- HTTP defines how Web clients request objects (e.g., Web pages) from Web servers and how servers transfer them to clients.
- When a user requests an object (for example, a Web page by clicking on a hyperlink), the **browser** sends HTTP request messages for the **objects in the page** to the server.
- The server receives the requests and responds with HTTP response messages that contain the **objects**.
- HTTP is **continuously evolving** to match the requirements (reliability vs. performance) of modern networking.

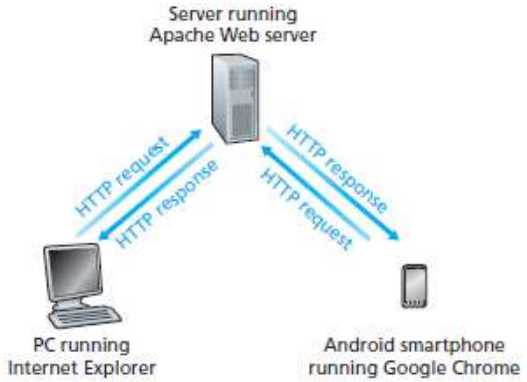




Application Layer
HTTP

Client -- HTTP request / method --> Server

Client <-- HTTP response --------- Server



HTTP Request Methods

- GET — retrieve a resource
- POST — submit data to a resource, often creating or triggering something.
- PUT — create or replace the resource at a known URI.
- DELETE — remove a resource.
- HEAD — like GET, but returns headers only.
- PATCH — partially modify a resource.

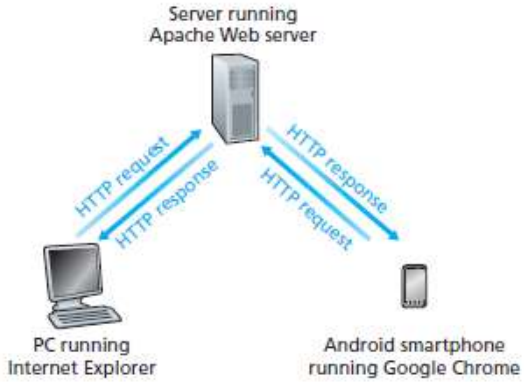
Safe methods: intended not to modify server state, e.g. GET, HEAD.

Idempotent methods: repeating them has the same intended effect as executing them once, e.g. GET, PUT, DELETE

Application Layer
HTTP

Client -- HTTP request / method --> Server

Client <-- HTTP response --------- Server



HTTP Request

METHOD resource HTTP-version

headers

optional body

```
GET /index.html HTTP/1.1
Host: example.com
```

```
PUT /products/42 HTTP/1.1
Host: example.com
Content-Type: application/json

{"name":"Router","price":99}
```

HTTP Response

HTTP-version status-code reason

headers

optional body

```
HTTP/1.1 200 OK
Content-Type: text/html
<html>
<body>
Hello
</body>
</html>
```

```
HTTP/1.1 201 Created
```

Application Layer
HTTP: Transport Protocol

- HTTP mainly uses **TCP as its underlying transport protocol**, but from 2022 (version HTTP/3) it **may also use UDP**.
  - Around 30% of the overall HTTP traffic is on UDP [Wikipedia, 2024].

- The HTTP client **first initiates a connection** with the server. Once the connection is established, the browser and the server processes will communicate through their **socket interfaces**.

- The client sends HTTP request messages into its socket interface and receives HTTP response messages from its socket interface. Similarly, the HTTP server receives request messages from its socket interface and sends response messages into its socket interface.

Application Layer

HTTP: Transport Protocol

- HTTP historically used TCP as a transport protocol because there are **several useful services are provided by TCP**, most of all **reliable data transfer**.
  - Recall reliability: TCP guarantees HTTP request/response messages to eventually arrive intact at the destination (if possible).

- However, **TCP services may slow down the communication**, hence the **recent implementation of HTTP uses also UDP** as transport protocol:
  - It is estimated to be around 30% faster than TCP-only communication.
  - It relies on **QUIC** (Quick UDP Internet Connection) protocol **that implements reliability on top of UDP**.

- Here we see one of the **great advantages of a layered architecture**: HTTP-based applications need not worry about reachability, data loss, etc.
  - HTTP applications can be **simpler** (both logically and computationally), by **delegating** most of the work to the lower-level stack.

Application Layer

HTTP: Statelessness

- Simplicity is very important for HTTP-based servers dealing with **large number of requests per second.**
  - In average, a server runs *around 1000 HTTP requests per second* (large search engine infrastructures receive hundreds of thousands of *queries per second!)*.

- HTTP applications are typically **stateless**: the server does not maintain any information about the interaction (sequence of requests/responses) with a specific client.
  - Example: if a client asks for the **same object twice** in a row, the server does not consider this request redundant, but **it just resends the object**, as it has completely forgotten the past.

- Statelessness is a property of the protocol, several applications are instead **stateful**.

Application Layer

HTTP: Persistent and Non-persistent Connections

- In many applications (such as HTTP-based), the client and server may communicate for an **extended period of time**, by sending **multiple pairs of request-response**.
  - This flow of messages may be made back-to-back, periodically (at regular intervals), or intermittently.

- **The choice:** when relying on a connection-oriented protocol (such as TCP or QUIC), we can have:
  - **Persistent connections**: all requests and their corresponding responses are sent over the same connection.
  - **Non-persistent connections**: for each request-response pair a new connection is established.

- HTTP/1.0 was non-persistent by default; today the default is persistent, and both ends can be configured otherwise.

Application Layer

HTTP: Persistent and Non-persistent Connections

- **Non-persistent connections**: for each request-response pair a new connection is established (HTTP/1.0).

```
TCP connection #1
    GET /index.html
    200 OK + HTML
close

TCP connection #2
    GET /foto.jpg
    200 OK + JPEG
close
```

```
TCP connection
    |
    +--- one request
    +--- one response
    |
   CLOSE
```

Application Layer

HTTP: Persistent and Non-persistent Connections

- **Non-persistent connections**: for each request-response pair a new connection is established (HTTP/1.0).
- **Persistent connections**: all requests and their corresponding responses are sent over the same connection (HTTP/1.1)
  - multiple HTTP requests can reuse the same TCP connection
  - browsers often open several parallel TCP connections to the same server

HTTP/1.1 → persistent TCP connections
HTTP/2 → multiplexed streams over one TCP connection
HTTP/3 → multiplexed streams over QUIC/UDP

Quick UDP Internet Connection runs implements reliable transport mechanisms on top of UDP

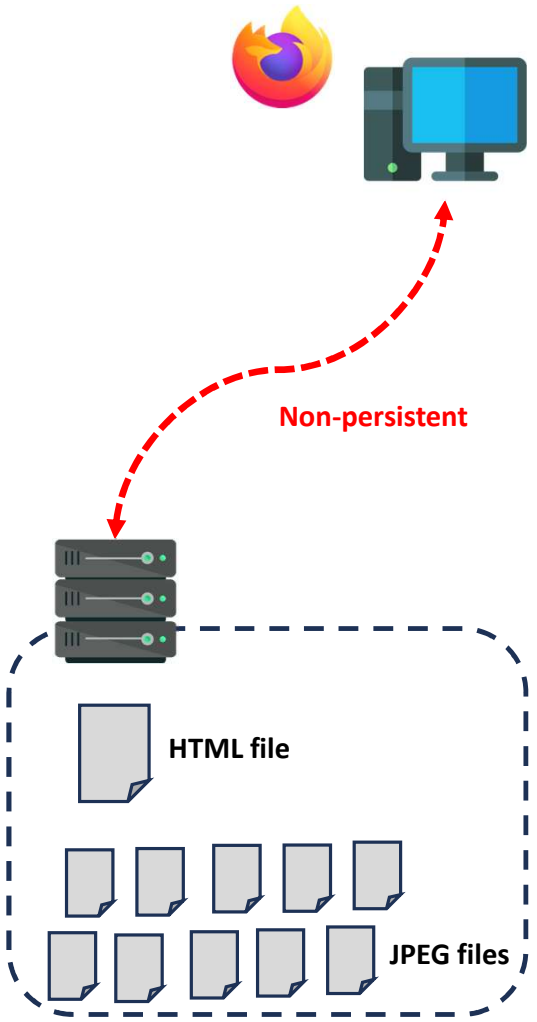
Application Layer

HTTP: Non-persistent Connection (Example)

- In a non-persistent connection, the connection is closed every time **a request is served** (e.g., an object is sent).
  - We will see HTTP examples on TCP.

- **Example:** assume to ask for a Web page consisting of a base HTML file and 10 JPEG images (all 11 of these objects reside on the same server).

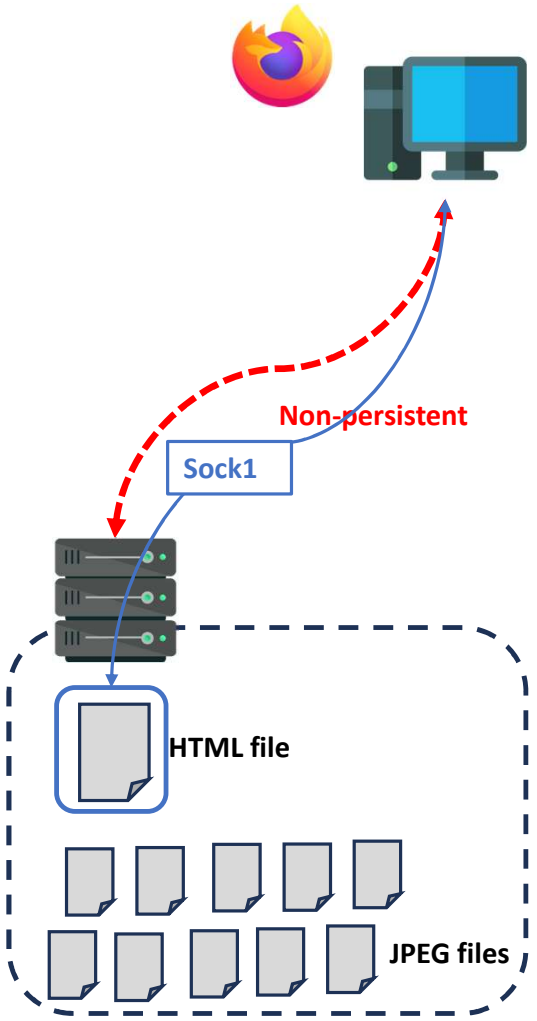
- The URL for the base HTML file is:
  - http://www.someschool.edu/someDepartment/home.index



Application Layer

HTTP: Non-persistent Connection (Example)

- The connection takes place as follows (with TCP):
  1. The HTTP **client process initiates a TCP connection** to the server www.someschool.edu on port number 80 (default HTTP port), we will have two sockets: one at the client and one at the server.
  2. The HTTP **client sends an HTTP request message** to the server via its socket including the path name /someDepartment/home.index.
  3. The HTTP **server process receives the request message** via its socket, **retrieves** the object /someDepartment/home.index (from RAM or disk), **encapsulates** the object in an HTTP response message, and **sends back** the response message to the client via its socket.
  4. The HTTP **server process tells TCP to close** the TCP connection, the TCP will (later) terminate the connection once it is sure that the client has received the response message intact (reliability).
  5. The HTTP **client receives the response message**. The TCP **connection terminates**. The message indicates that the encapsulated object is an HTML file. The client extracts the file from the response message, examines the HTML file, and **finds references to the 10 JPEG objects**.
  6. The first four **steps are then repeated** for each of the referenced JPEG objects.



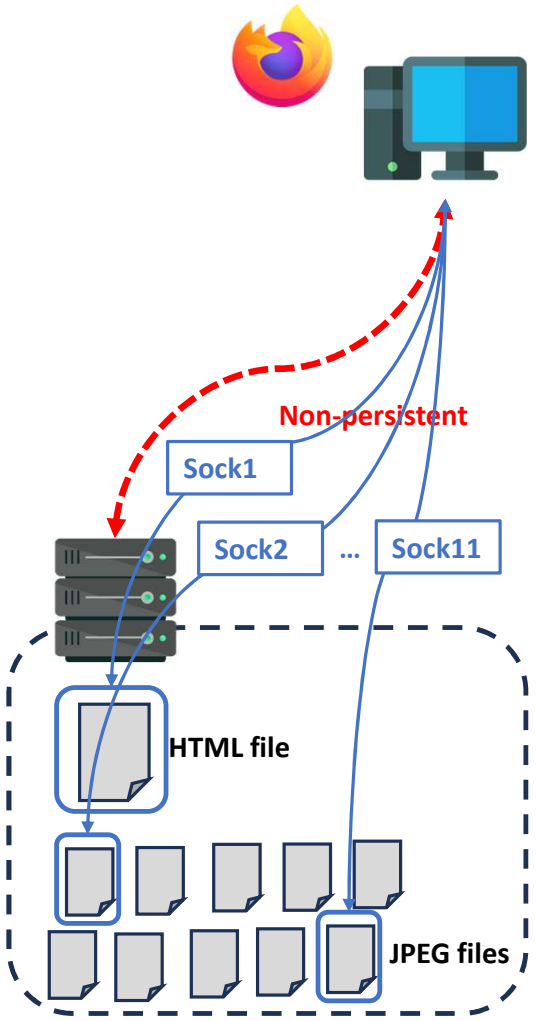
Application Layer

HTTP: Non-persistent Connection (Example)

- Since the connection does not persist over different objects, we need **11 TCP connection** (and 11 pairs of sockets) to transfer the whole page, hence the elements of a page may arrive in **different moments**.

- Note: HTTP **defines only the communication protocol** between the client HTTP program and the server HTTP program, not how contents are displayed.

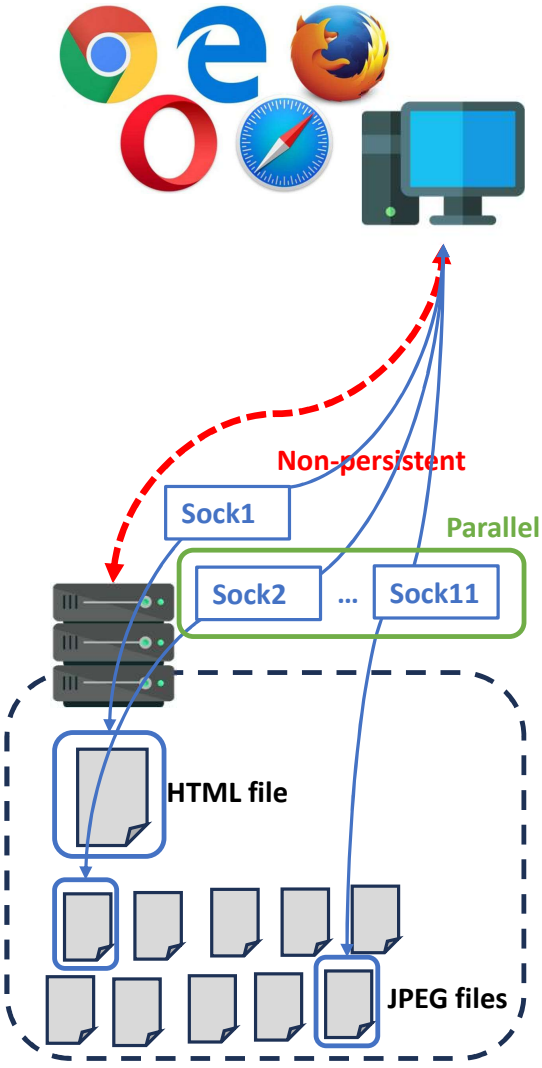
- The way web pages are displayed **depends on the browser**: two different browsers may interpret (hence, display to the user) a Web page in somewhat different ways.
  - E.g., waiting for the whole page to be loaded before to show it, showing it step by step, etc.



Application Layer

HTTP: Non-persistent Connection (Example)

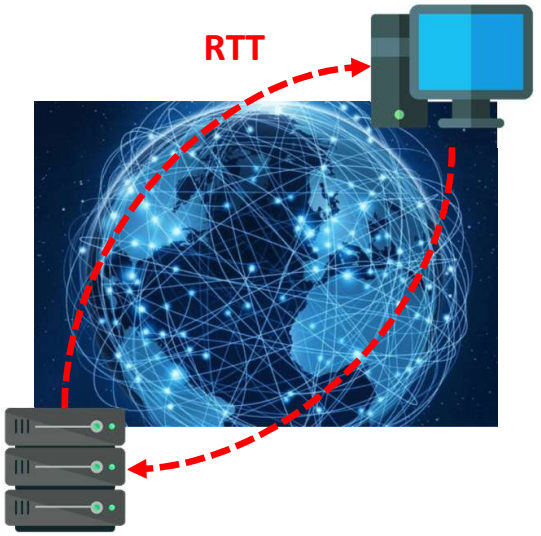
- The different **connections** may be established in **sequence** or in **parallel**.
  - In this example, once we know that 10 JPEG images are needed (i.e., the main page is received), we may download all of them in parallel.
- In modern browsers users may also **configure** the degree of parallelism involved:
  - Most browsers open 5 to 10 parallel TCP connections, and each of these connections handles one request-response transaction.
  - Clearly, the use of parallel connections **shortens the response time but increases the computational cost**.
- Despite parallelism, establishing multiple (TCP) connections may still induce a significant **time overhead.**



Application Layer

HTTP: Non-persistent Connections (RTTs)

- It is **difficult to precisely estimate times** on Internet. We can estimate the overhead it takes to finalize a HTML request in terms of **round-trip times** (RTTs).



- The **RTT** is the time it takes for a small packet to travel from client to server and back to the client.

- When a user clicks on a hyperlink the browser initiates a connection between with the Web server.

- Assuming a TCP-based request (connection-oriented), to ensure the connection a **three-way handshake** is performed…

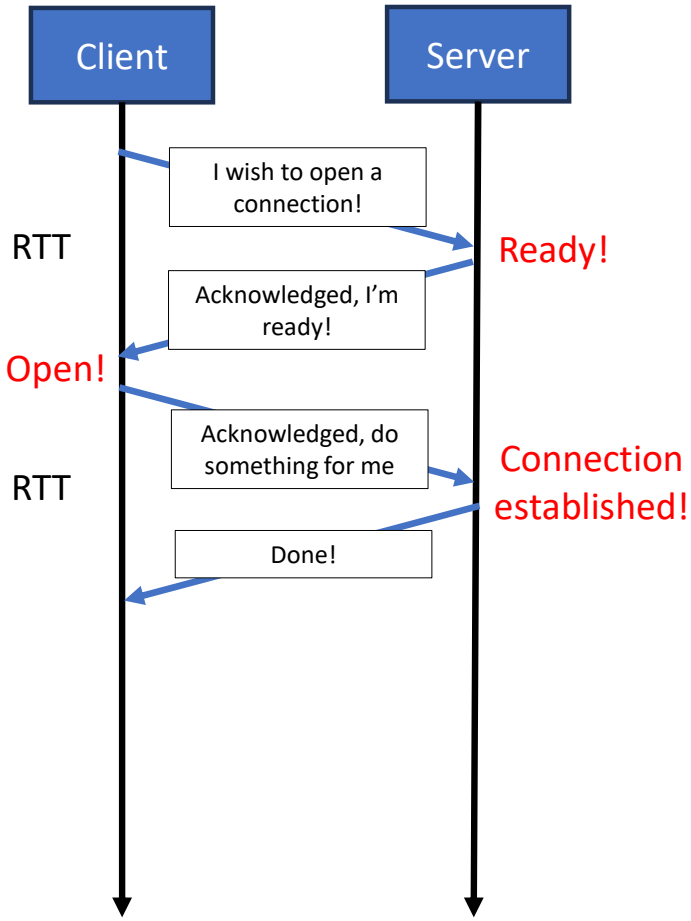
Application Layer

HTTP: Non-persistent Connections (RTTs)

- TCP **three-way handshake** procedure is performed through the following 3 steps:
  - The client sends a small TCP segment to the server, to signal that a **connection is requested**.
  - The **server acknowledges** and responds with a small TCP segment.
  - The client acknowledges back to the server **and in HTTP the request rides on this third segment.**

- So the first two parts take one RTT, and that is what opening a connection costs before any data can flow.

- Notice that a similar handshake also takes place when **connection is closed**.



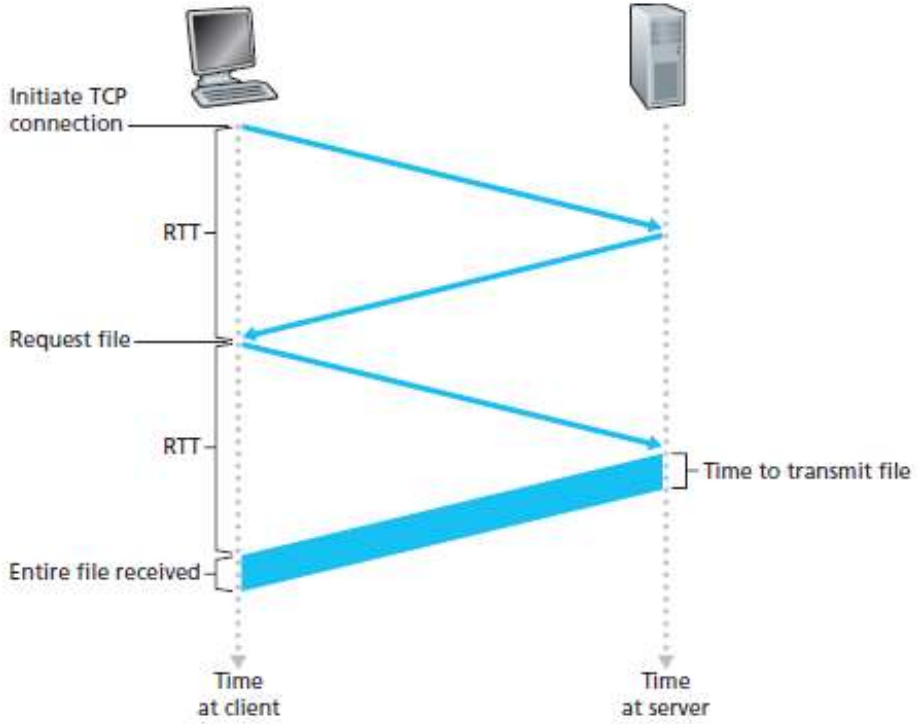
Application Layer

HTTP: Non-persistent Connections (RTTs)

- The first two parts of the three-way handshake take **one RTT**.
- In HTTP, the client sends the HTTP **request message** combined with the third part of the three-way handshake (the acknowledgment).
- Once the request message arrives, the server sends the HTML file. This HTTP request/response eats up **another RTT**.
- Roughly, the total response time **is two RTTs plus the transmission time** at the server of the HTML file:

$$T \approx 2RTT + T_{tx}$$

$$T_{tx} = \frac{L}{R}$$

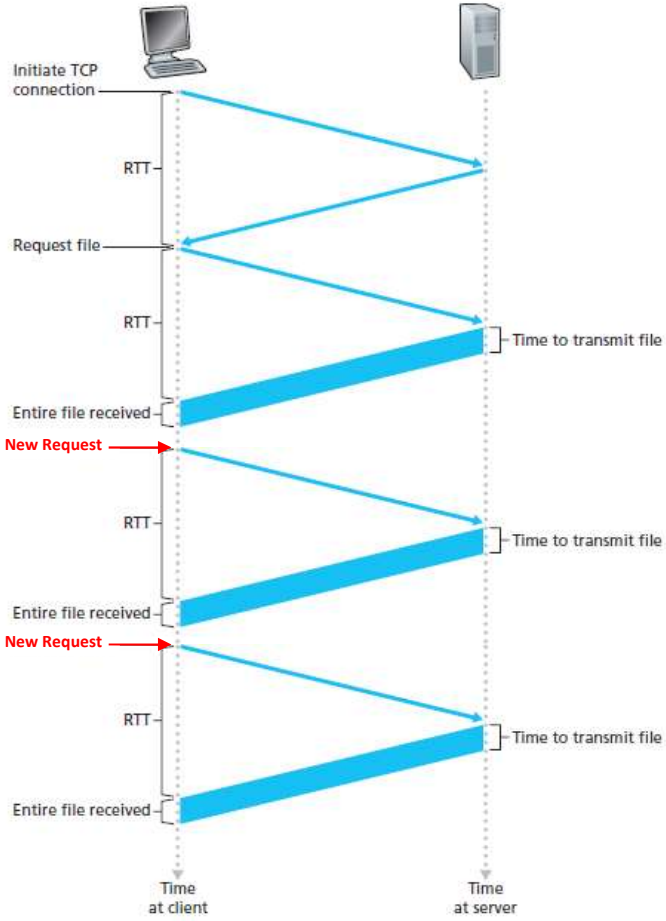


Application Layer

HTTP: Persistent Connections

- In persistent connections, the server **leaves open** the TCP connection after sending a response.
- Requests and responses between the same client and server can be sent over the same connection: **an entire Web page** (the HTML file and the 10 images from the previous example) can be sent over **a single persistent TCP connection**.
- **Multiple Web pages** residing on the same server can also be sent from the server to the same client over a single persistent TCP connection.
- In the simplified HTTP timing model, the main saving is about one RTT per additional object:

$$\text{next object} \approx 1RTT + T_{tx}$$



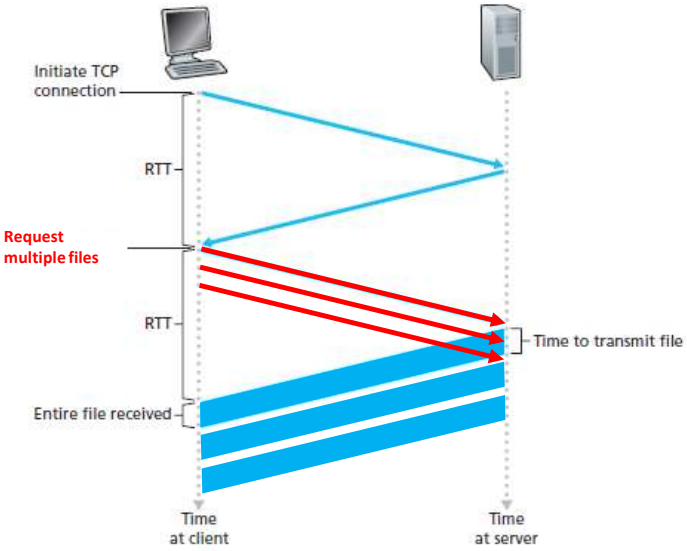
Application Layer

HTTP: Persistent Connections with Pipelining

- **HTTP/1.1 Pipelining**: multiple requests may be sent without waiting for previous responses, but responses must be returned in request order.

GET A ->
GET B ->
GET C ->

<- response A
<- response B
<- response C



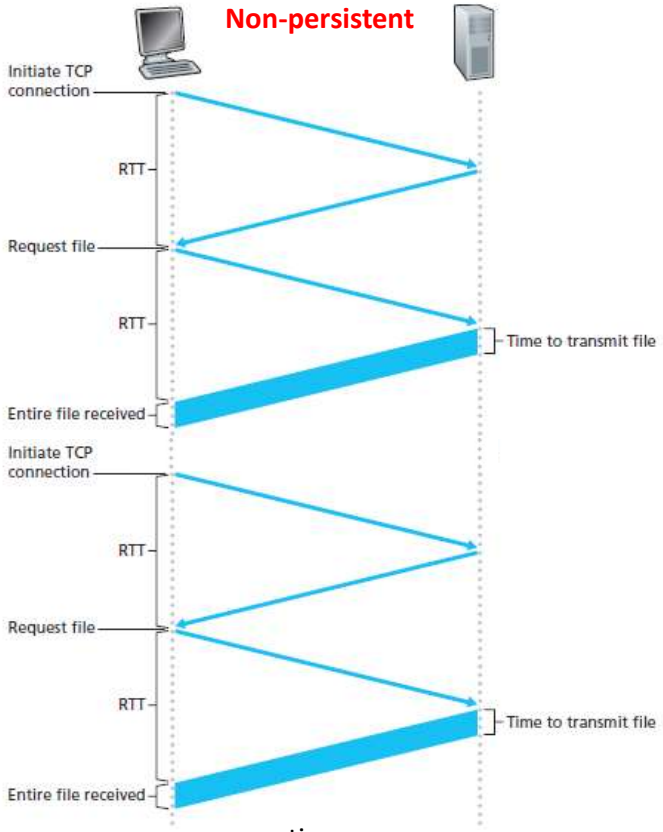
- **HTTP/2 multiplexing:** multiple independent request-response streams can be interleaved over the same TCP connection.

GET1 GET2 GET3

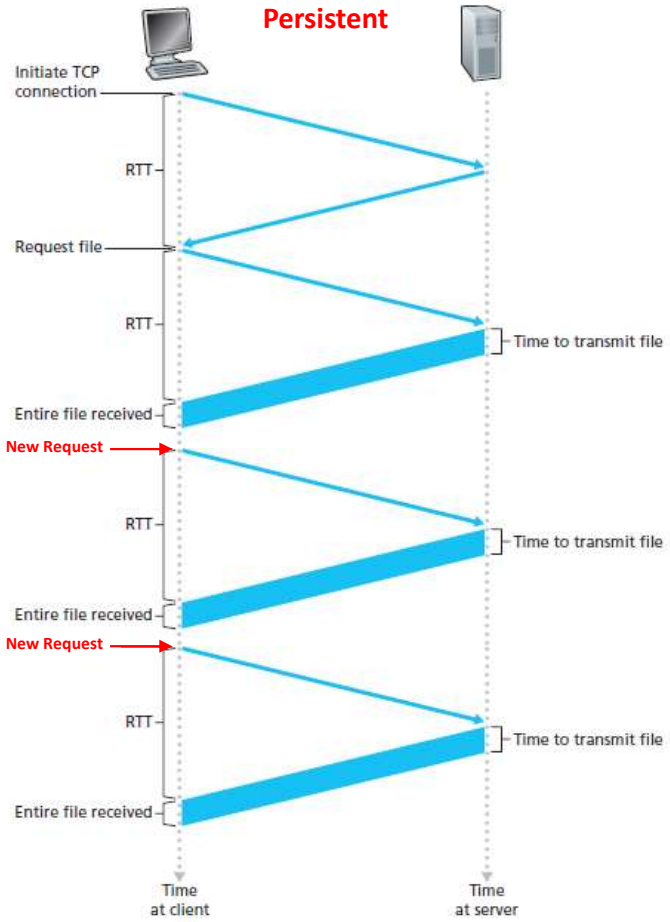
R2
R3
R1

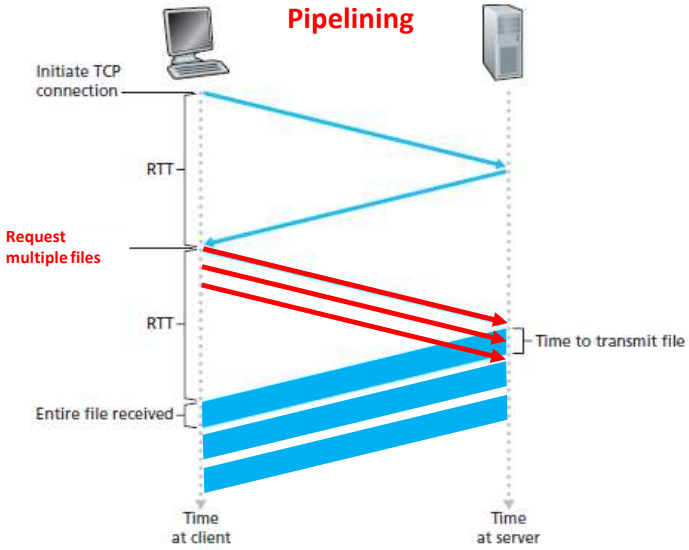
Application Layer

HTTP: Persistent vs. Non-persistent Connections



continues…





Rough comparison of RRTs needed to request 3 files considering the 3 modalities

Application Layer

HTTP: Persistent vs. Non-persistent Connections

The eleven-object page, no losses, transmission time ignored

- Non-persistent, one at a time: 2 × 11 = **22 RTTs**.
- Non-persistent, 5 in parallel: 2 for the HTML (after 10 images are needed), then two rounds (5+5) of 2 for the images: **6 RTTs**.
- Persistent, no pipelining: 1 to open, then 1 per object: **12 RTTs**.
- Persistent with pipelining: 1 to open, 1 for the HTML, 1 for the ten images: **3 RTTs**.

Same page, same server, same distance. The difference comes from connection reuse, parallelism, and whether requests must wait for previous responses.

Application Layer

HTTP: Persistent vs. Non-persistent Connections

- **Persistent:**
  - Reuse the same TCP connection for multiple HTTP exchanges
  - Reduce repeated connection setup overhead
  - Require fewer TCP connection resources.
  - Connection may be left open even if unused. Typically, the HTTP server closes a connection when it isn't used for a certain time (timeout).
- **Non-persistent:**
  - Use a new TCP connection for each request-response exchange.
  - Require repeated TCP setup and teardown.
  - Consume more connection-related resources.
  - In the simplified timing model, each new object pays about one extra RTT for connection establishment.

Application Layer

HTTP: Persistent vs. Non-persistent Connections

- **Persistent:**
  - What if the server times out just as the client reuses the connection?
  - The server closes an idle persistent connection on its own, and the client is not asked.

Application Layer
HTTP: Persistent

- **Persistent:**
  - What if the server times out just as the client reuses the connection?
  - The server closes an idle persistent connection on its own, and the client is not asked.
  - If the connection is closed while the client is sending or waiting for a response, the request may fail (write() or read() failure). In some cases, the client cannot know whether the server processed the request before the connection was lost.

  - A client that reuses an idle connection must be ready to **open a new one and send the request again**.

  - Automatic retry is safe only for requests whose repetition has no additional effect. This is why **idempotent methods** are important; GET is also expected to be safe, i.e. not to modify server state

Application Layer
HTTP/2

- **Why does one big object delay ten small ones?**
  - A page with a large video near the top, many small objects below, one persistent connection, and a slow link somewhere on the path.
  - The video occupies the connection; the small objects are ready and wait behind it. **This is head-of-line blocking.**
  - Browsers work around it by opening up to six parallel connections — which also grabs a larger share of the bottleneck, since TCP gives each connection, not each page, an equal share.

HTTP/2 reduces the need for multiple parallel TCP connections by **multiplexing** several independent streams (HTTP/2 frame) over a single TCP connection

Application Layer
HTTP/2

- A framing sub-layer breaks every message into **small frames**:
  - A framing sub-layer breaks HTTP messages into binary frames. Header fields are carried in HEADERS frames, while the message body is carried in one or more DATA frames.
- Frames belonging to different responses are **interleaved** on the single connection, and reassembled at the other end.
- For example, a large response can be interleaved with several **small responses**, so the **small objects** do not have to wait for the large one to finish.
- HTTP methods, status codes, URLs, and header semantics remain **unchanged**; what changes is how messages are encoded and transported.

HTTP/2 therefore supports **multiple concurrent streams over a single TCP connection**, reducing application-level head-of-line blocking.

Application Layer
HTTP/2

- HTTP/2 lets clients **prioritize streams**, so critical resources can be sent first
  - E.g., higher priority to a style sheet or a script to render the page than to images below the visible area
  - Modern HTTP/2 uses a simpler priority model than the original weight/dependency scheme
- **Server Push** allowed servers to send resources before they were requested
  - Server Push is now rarely used and has been dropped by major browsers
  - it could waste bandwidth by sending resources that were already cached or not actually needed

Application Layer
HTTP/3 and QUIC

HTTP/3 is the latest major version of HTTP, designed to preserve HTTP semantics while replacing TCP with QUIC as the transport protocol:

- **QUIC is a transport protocol** implemented in user space, running over UDP:
- **QUIC adds reliability, multiplexed streams, congestion control, and per-stream/connection flow control** on top of UDP.
- **Connection setup is faster** because QUIC integrates TLS 1.3 (Transport Layer Security) with transport connection establishment (the connection and its encryption are negotiated together).
- Packet loss affecting one stream does not block unrelated streams, avoiding TCP's cross-stream head-of-line blocking.
- HTTP/3 runs over QUIC. Functions such as **multiplexing** and **flow control** are handled by QUIC rather than HTTP itself.

Application Layer
HTTP Summary

- **A Web page is a set of objects**, each with its own URL, and each fetched separately.
- HTTP defines the **messages** and the **rules of exchange**, and nothing about how a page is displayed.
- HTTP/1.1 and HTTP/2 run over **TCP** and inherit its reliable, ordered byte stream; HTTP/3 runs over **QUIC** (Quick UDP Internet Connection).
- **HTTP is stateless**: each request is self-contained from the protocol point of view, although applications can maintain state using cookies and sessions.
- **Non-persistent HTTP** typically uses one TCP connection per object, costing about two RTTs per object; **persistent HTTP** pays the handshake once, then reuses the connection.
- **Pipelining** removes waiting between requests; HTTP/2 **multiplexes streams**, while HTTP/3 carries HTTP over QUIC.

Application Layer
Wireshark

On Debian, Ubuntu and derivatives. Answer yes when asked whether non-root users may capture packets.

```
$ sudo apt update && sudo apt install wireshark
$ sudo dpkg-reconfigure wireshark-common    if you answered no
$ sudo usermod -aG wireshark $USER          then log out and back in
$ groups                                    wireshark must appear
$ getcap /usr/bin/dumpcap
cap_net_raw,cap_net_admin=eip               what it must print
```

If getcap prints nothing, assign those two capabilities to the binary yourself, with setcap.

sudo setcap cap_net_raw,cap_net_admin=eip /usr/bin/dumpcap

Application Layer

Wireshark on Windows

- Download and install Wireshark from the official website.
- During installation, install Npcap when prompted.
- Npcap provides access to the network interfaces and allows Wireshark to capture packets.
- Start Wireshark and select the active interface, typically Wi-Fi or Ethernet.
- Administrator privileges are normally not required for everyday captures if Npcap is correctly installed.

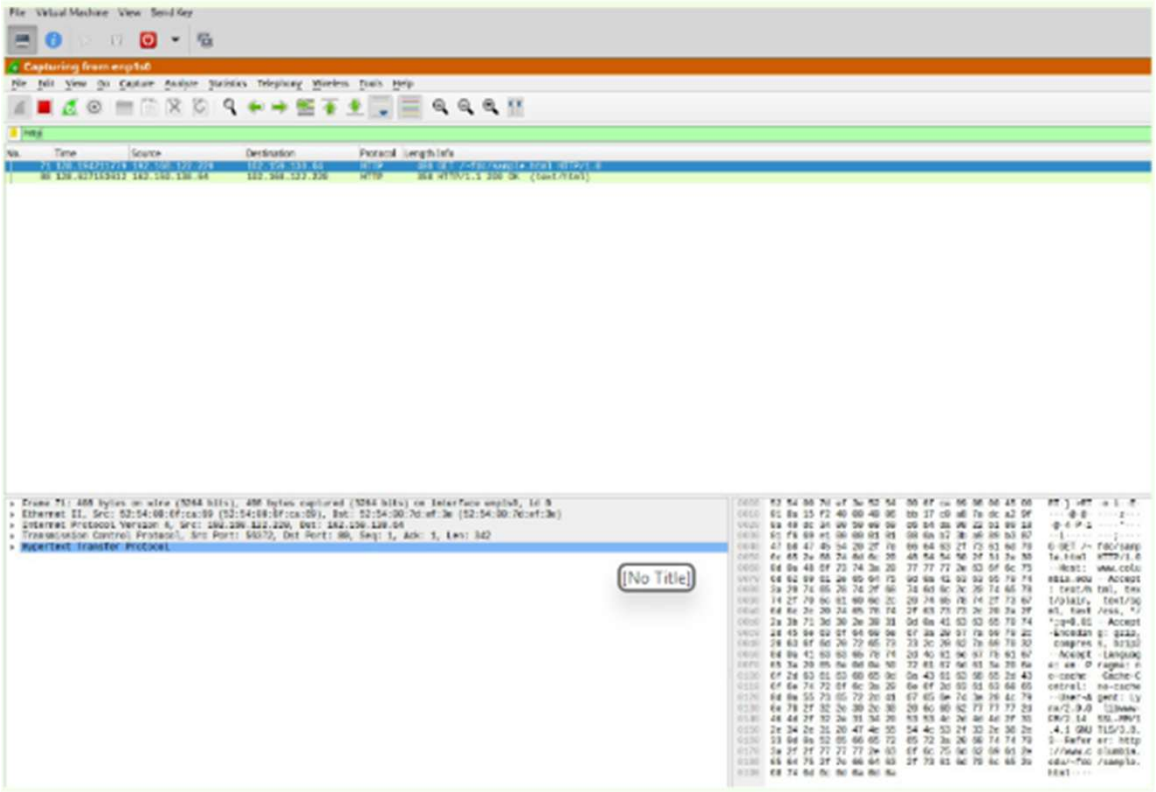
Application Layer
Wireshark on Windows

- Start Wireshark as set up on the previous slide, without starting the capture.
- Type http in the display filter, so that only HTTP packets are shown.
- Wait a little over a minute — you will find out why — then start capturing.
- Fetch a very simple page (curl – client url):
  - $ curl http://www.columbia.edu/~fdc/sample.html (in linux)
  - $ curl.exe http://www.columbia.edu/~fdc/sample.html (in windows)
- Stop the capture.
- Collapse the Frame, Ethernet, IP and TCP details, and expand the HTTP one

Application Layer

Wireshark on Windows

- What you should see
- Two HTTP packets: the GET your client sent and the 200 OK that came back — sitting inside a TCP segment, an IP datagram and an Ethernet frame.



Application Layer

Wireshark on Windows

(a) Which HTTP version does your client announce? And which one does the server answer with?

(b) Which languages does the client tell the server it is willing to accept?

(c) What is the IP address of your machine, and what is the server's?

The answers are all in the two packets you have just captured — read them off the capture, not off these slides.

Application Layer
Wireshark on Windows

(d) What status code did the server return?

(e) When was the page last modified on the server?

(f) How many bytes of content were sent to your client?

(g) In the raw bytes of the packet, are there header lines that the packet list does not show?

On (e): the file may look modified less than a minute before you fetched it.

That server refreshes the timestamp every minute, so that everyone downloads a fresh copy.

Then repeat with http://scottaaronson.com and again with a modern browser instead of lynx: what changed, and why?

Application Layer

Wireshark on Windows

(d) What status code did the server return?

(e) When was the page last modified on the server?

(f) How many bytes of content were sent to your client?

(g) In the raw bytes of the packet, are there header lines that the packet list does not show?

On (e): the file may look modified less than a minute before you fetched it. That server refreshes the timestamp every minute, so that everyone downloads a fresh copy. Then repeat with http://scottaaronson.com, and again with a modern browser instead of lynx: what changed, and why?